

HOTEL ROOM BOOKING & RESERVATION PLATFORM

CIA-3 Project Development — 5th Semester

Project Code: P03

Section / Batch: **[ENTER SECTION / BATCH]**

TEAM DETAILS

S.No	Student Name	Roll No.	Department	Section
1	[ENTER NAME]	[ENTER ROLL NO.]	[ENTER DEPARTMENT]	[ENTER SECTION]
2	[ENTER NAME]	[ENTER ROLL NO.]	[ENTER DEPARTMENT]	[ENTER SECTION]
3	[ENTER NAME]	[ENTER ROLL NO.]	[ENTER DEPARTMENT]	[ENTER SECTION]
4	[ENTER NAME]	[ENTER ROLL NO.]	[ENTER DEPARTMENT]	[ENTER SECTION]

GitHub Repository

[PASTE COMPLETE GITHUB REPOSITORY URL HERE]

Before submission, replace all placeholders on this page with official team details and the final repository URL. The repository must be public or shared with the faculty coordinator if private.

1. PROJECT OVERVIEW

The Hotel Room Booking & Reservation Platform is a Node.js, Express.js and MongoDB backend for hotel property management and end-to-end room reservation operations. It supports secure guest authentication, hotel and room inventory, date-and-occupancy availability, reservation pricing, booking lifecycle management, staff operations, cancellation/refund handling, invoice summaries and administrative reporting.

The implementation follows an MVC-style backend organization. Routes expose REST APIs, controllers implement business logic, models define MongoDB schemas, and middleware provides JWT authentication, role authorization, request validation and centralized error handling.

The primary objective is to demonstrate a complete working backend workflow with server-side business rules rather than unrestricted CRUD operations. The application is intended to be demonstrable through Postman and a local Node.js server connected to MongoDB.

Technology Stack

Technology	Purpose
Node.js	Backend runtime
Express.js	REST API framework
MongoDB + Mongoose	Database and ODM
JWT	Authentication and protected routes
bcrypt	Password hashing
Postman	API testing/documentation

Scope

Payment gateways, SMS/email providers and maps are outside the required backend scope. A Postman-only demonstration is acceptable. The project assumes a single currency/time-zone unless an extension is documented.

2. IMPLEMENTED MODULES SUMMARY

The implementation covers the 13 mandatory P03 hotel-booking modules.

No.	Module	Implemented capability
1	Guest Registration & Authentication	Registration/login, bcrypt password hashing and JWT authentication.
2	Hotel & Property Management	Admin hotel CRUD: name, city, amenities and rating.
3	Room Type & Inventory Management	Room types, capacity, base price, total rooms and individual rooms.
4	Availability Search Engine	Search by hotel, dates and occupancy; excludes unavailable inventory.
5	Reservation Booking Workflow	Date validation and availability re-check before reservation creation.
6	Dynamic Pricing Rules	Date/season/weekend multipliers applied during pricing.
7	Booking Status Management	Controlled reservation lifecycle and restricted cancellation.
8	Check-in / Check-out	Staff/Admin operational check-in and check-out timestamps.
9	Housekeeping Status Tracking	Ordered room state: DIRTY → CLEANING → CLEAN.
10	Cancellation & Refund Policy Engine	Refund based on time remaining before check-in.
11	Guest Booking History	Guest-specific past and upcoming bookings with ownership checks.
12	Invoice Generation Summary	Stay cost, pricing, add-ons and taxes summarized for a booking.
13	Admin Occupancy Reports	Occupancy and revenue reports for administrative review.

3. SYSTEM ARCHITECTURE & WORKFLOW

The backend uses an MVC-style structure. A request is received by an Express route, passes through authentication/authorization and validation middleware where required, reaches a controller for business logic, accesses a Mongoose model, and returns a JSON response. Centralized error handling catches application errors and formats consistent responses.

Repository structure

```
project-root/
  ■■■ config/ MongoDB connection
  ■■■ models/ Mongoose schemas
  ■■■ routes/ Express routes
  ■■■ controllers/ Business logic
  ■■■ middleware/ JWT, validation, error handling
  ■■■ scripts/ Admin seed script
  ■■■ postman/ Exported Postman collection
  ■■■ server.js Application entry point
  ■■■ package.json
  ■■■ README.md
  ■■■ .env.example
  ■■■ .gitignore
```

Authentication workflow

Guest registration → bcrypt password hashing → MongoDB user record → login credential verification → JWT issuance → Bearer token on protected requests → JWT verification → role-based authorization.

Booking workflow

Select hotel/room type → enter check-in/check-out and occupancy → validate dates → check room availability → apply dynamic pricing → calculate nights, taxes and add-ons → create reservation → confirm through controlled status transition.

Staff workflow

Staff/Admin checks in a valid booking → room becomes occupied → checkout records actual time → room becomes DIRTY → housekeeping moves DIRTY → CLEANING → CLEAN.

Reservation conflict rule

$\text{requestedCheckIn} < \text{existingCheckOut}$ AND $\text{requestedCheckOut} > \text{existingCheckIn}$ means the requested reservation overlaps an existing booking and must be rejected.

4. DATABASE SCHEMA & RELATIONSHIPS

MongoDB collections are modelled as independent resources and connected with Mongoose ObjectId references where relationships exist. This keeps hotel, room inventory, pricing and booking data independently manageable.

Collection	Key fields	Purpose / relationship
users	name, email, passwordHash, role	Authentication identity; guest referenced by bookings.
hotels	name, city, amenities[], rating	Hotel property; parent for room types.
roomTypes	hotelId, name, basePrice, totalRooms, capacity	Sellable room category under a hotel.
rooms	hotelId, roomTypeId, roomNumber, status, housekeepingStatus	Physical room inventory.
bookings	guestId, hotelId, roomTypeId, roomId, checkIn, checkOut, status, totalAmount	Reservation and lifecycle record.
pricingRules	hotelId/roomTypeId, type, date range, multiplier, active	Rules used for dynamic pricing.

Collection relationship diagram

```
users ■■■■■■■■ bookings ■■■■■■■■ hotels
■ ■
■■■■■■■ roomTypes ■■■■ rooms
■
■■■■■■■ pricingRules
```

Schema and indexing considerations

The README documents deliberate references between independently managed resources and records indexes for common relation lookups such as users by email, roomTypes by hotelId, rooms by roomTypeId and bookings by guestId. The design avoids unnecessary relational-style joins.

5. API REFERENCE & SAMPLE DEMONSTRATION

The repository README documents 26 endpoints and the exported Postman collection provides requests for the implemented API surface.

Method	Endpoint	Description
POST	<code>`/api/auth/register`</code>	Guest registration
POST	<code>`/api/auth/login`</code>	Login and JWT
GET	<code>`/api/hotels`</code>	List hotels
POST	<code>`/api/admin/hotels`</code>	Admin creates hotel
PUT	<code>`/api/admin/hotels/:id`</code>	Admin updates hotel
DELETE	<code>`/api/admin/hotels/:id`</code>	Admin deletes hotel
POST	<code>`/api/admin/room-types`</code>	Create room type
GET	<code>`/api/room-types`</code>	List room types
POST	<code>`/api/admin/rooms`</code>	Create room
GET	<code>`/api/rooms`</code>	List rooms
GET	<code>`/api/hotels/search`</code>	Search room availability
POST	<code>`/api/bookings`</code>	Create reservation
GET	<code>`/api/bookings/:id`</code>	Get booking
PUT	<code>`/api/bookings/:id/status`</code>	Controlled status transition
PUT	<code>`/api/bookings/:id/cancel`</code>	Cancel + refund
GET	<code>`/api/guests/:id/bookings`</code>	Guest booking history
POST	<code>`/api/admin/pricing-rules`</code>	Create pricing rule
GET	<code>`/api/pricing-rules`</code>	List pricing rules
PUT	<code>`/api/admin/pricing-rules/:id`</code>	Update pricing rule
DELETE	<code>`/api/admin/pricing-rules/:id`</code>	Delete pricing rule
PUT	<code>`/api/bookings/:id/checkin`</code>	Staff/Admin check-in
PUT	<code>`/api/bookings/:id/checkout`</code>	Staff/Admin check-out
PUT	<code>`/api/rooms/:id/housekeeping`</code>	Staff/Admin housekeeping update
GET	<code>`/api/bookings/:id/invoice`</code>	Invoice summary
GET	<code>`/api/admin/reports/occupancy`</code>	Occupancy report
GET	<code>`/api/admin/reports/revenue`</code>	Revenue report

Sample API demonstration

Registration: POST `/api/auth/register`

```
{ "name": "Demo Guest", "email": "guest@example.com", "password": "Example@123" }
```

Availability: GET `/api/hotels/search` — provide hotel/date/occupancy criteria and verify unavailable inventory is excluded.

Booking: POST `/api/bookings` — submit a valid room/date selection and verify the reservation is created only after availability validation.

Operational: PUT `/api/bookings/:id/checkin` and PUT `/api/bookings/:id/checkout` — demonstrate role restriction and lifecycle updates.

Reports: GET /api/admin/reports/occupancy and GET /api/admin/reports/revenue — demonstrate administrative aggregation output.

6. SECURITY, VALIDATION & TESTING

Security

Passwords are stored as bcrypt hashes under passwordHash. JWT Bearer tokens protect authenticated routes. Admin and Staff operations are role-restricted, preventing unrestricted access to management and operational actions.

Validation and error handling

Request validation is performed before controller business logic. The centralized Express error handler returns consistent JSON error responses. Invalid identifiers, missing fields, invalid dates, unauthorized requests and workflow conflicts are handled as API errors instead of causing unhandled server crashes.

Postman test checklist

Test	Expected result
Happy path	Registration/login and a valid core operation succeed.
Validation failure	Missing/invalid required field returns 400.
Authentication failure	Protected route without valid token returns 401.
Authorization failure	Wrong role returns 403.
Business-rule conflict	Overlapping booking or invalid status transition is rejected.
Not found	Invalid/non-existent ID returns a clean 4xx response.

Local setup

```
npm install
Copy-Item .env.example .env
npm run seed-admin
npm start
```

The project does not require mongosh when the Node.js application connects directly to MongoDB. Do not commit .env or node_modules.

7. GITHUB SUBMISSION, LEARNINGS & CONCLUSION

GitHub readiness checklist

- Create the repository and keep it public, or share it with the faculty coordinator if private.
- Upload complete source code, README.md and the exported Postman collection.
- Do not upload node_modules or .env.
- Keep .gitignore and .env.example in the repository.
- Add all team members as collaborators.
- Each member should make genuine commits from their own GitHub account so contribution history is visible.
- Put the final GitHub URL directly below the Team Details table on Page 1 of this report.

Learnings

The project provides practical experience in Express MVC organization, Mongoose schema design, JWT authentication, bcrypt password hashing, role-based authorization, server-side validation, centralized error handling and business-rule enforcement. Date-range conflict logic and controlled lifecycle transitions are especially important because they demonstrate domain logic beyond simple CRUD.

Challenges addressed

The implementation addresses overlapping reservation prevention, re-checking availability at booking time, dynamic pricing, staff/admin protection, housekeeping state order, cancellation/refund calculation and consistent API errors.

Conclusion

The resulting backend integrates the 13 mandatory hotel-booking modules into one demonstrable workflow. Before final LMS submission, the team should replace the Page 1 placeholders, paste the final GitHub URL, capture required code/output screenshots for the PDF and PPT, and verify that the repository and demo match the submitted report.

Required filename format: ProjectCode_TeamNo_Section.pdf — example: P03_Team03_A.pdf.